

Eagle Eye

Technical documentation

From contract files to structured intelligence.
Architecture, extraction, data models and interfaces.

01

INGEST

Files and folders

02

EXTRACT

Local models

03

CONNECT

Parties and conflicts

Flask / Svelte 5 + TypeScript / PostgreSQL

Locally hosted contract-ingestion prototype

Contents

Backend architecture	3
Processing flow	5
Database schema	7
Backend module reference	13
HTTP API	17
Frontend calendar	18
Frontend contracts view	19
Frontend architecture	20
Local models and privacy	22
Prototype limitations	23

About this edition

This handbook presents the full DOCS.md reference with vector diagrams and print-oriented tables. The Markdown file remains the source of truth. Model outputs and confidence scores require human review.

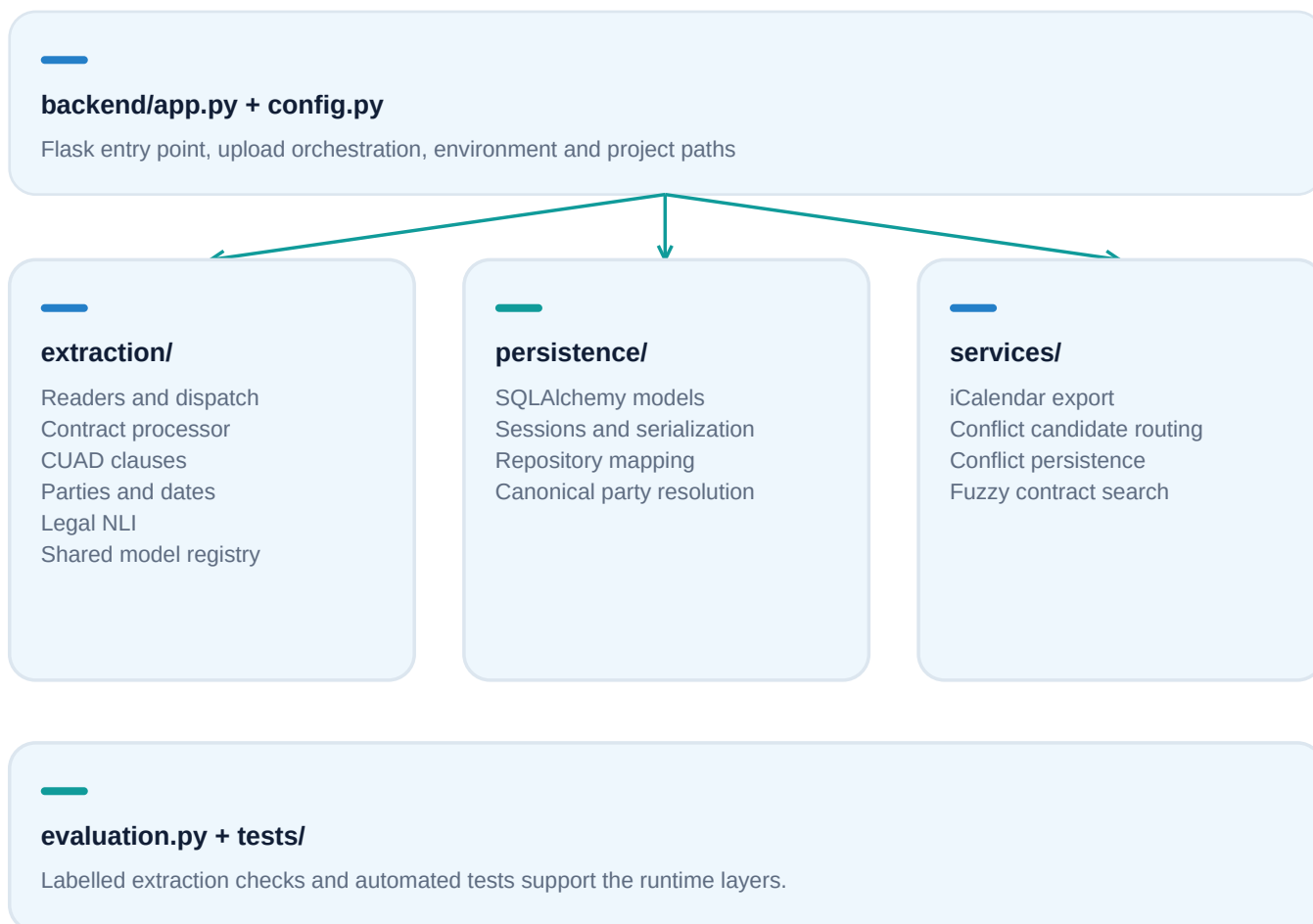
Visual direction: AITHENA website - <https://www.aithena.sg/>

Eagle Eye is a locally hosted contract-ingestion prototype. A user selects one or more contracts or a folder in the browser; the backend extracts text, CUAD clauses, parties, explicit dates, and grounded fixed-term expiration dates, then checks eligible clause pairs for possible contradictions and stores queryable records in PostgreSQL.

01 / TECHNICAL REFERENCE

Backend architecture

The backend is one Python package, grouped by responsibility:



Module inventory

backend/	
app.py	Flask factory, routes, and upload handling
config.py	environment and project paths
evaluation.py	labelled extraction evaluation runner
extraction/	
contract_processor.py	extraction pipeline coordinator
clause_extractor.py	CUAD clause extraction
conflict_extractor.py	bidirectional legal NLI scoring
date_extractor.py	explicit and derived dates
party_extractor.py	party detection and normalization
model_registry.py	lazy model loading
readers/	
text_formatter.py	file-type dispatch
png_reader.py	PNG and JPEG OCR
pdf_reader.py	embedded PDF text
docx_reader.py	DOCX paragraphs and tables
persistence/	
database.py	SQLAlchemy models and sessions
contract_repository.py	extraction-to-database mapping
party_resolution.py	canonical party resolution
services/	
calendar_export.py	iCalendar generation
conflict_analysis.py	candidate routing and persistence
contract_search.py	fuzzy text search

Package imports are absolute from backend, so every dependency identifies the layer that owns it. The root directory contains project configuration, documentation, the frontend, tests, and evaluation data rather than runtime Python modules.

Processing flow

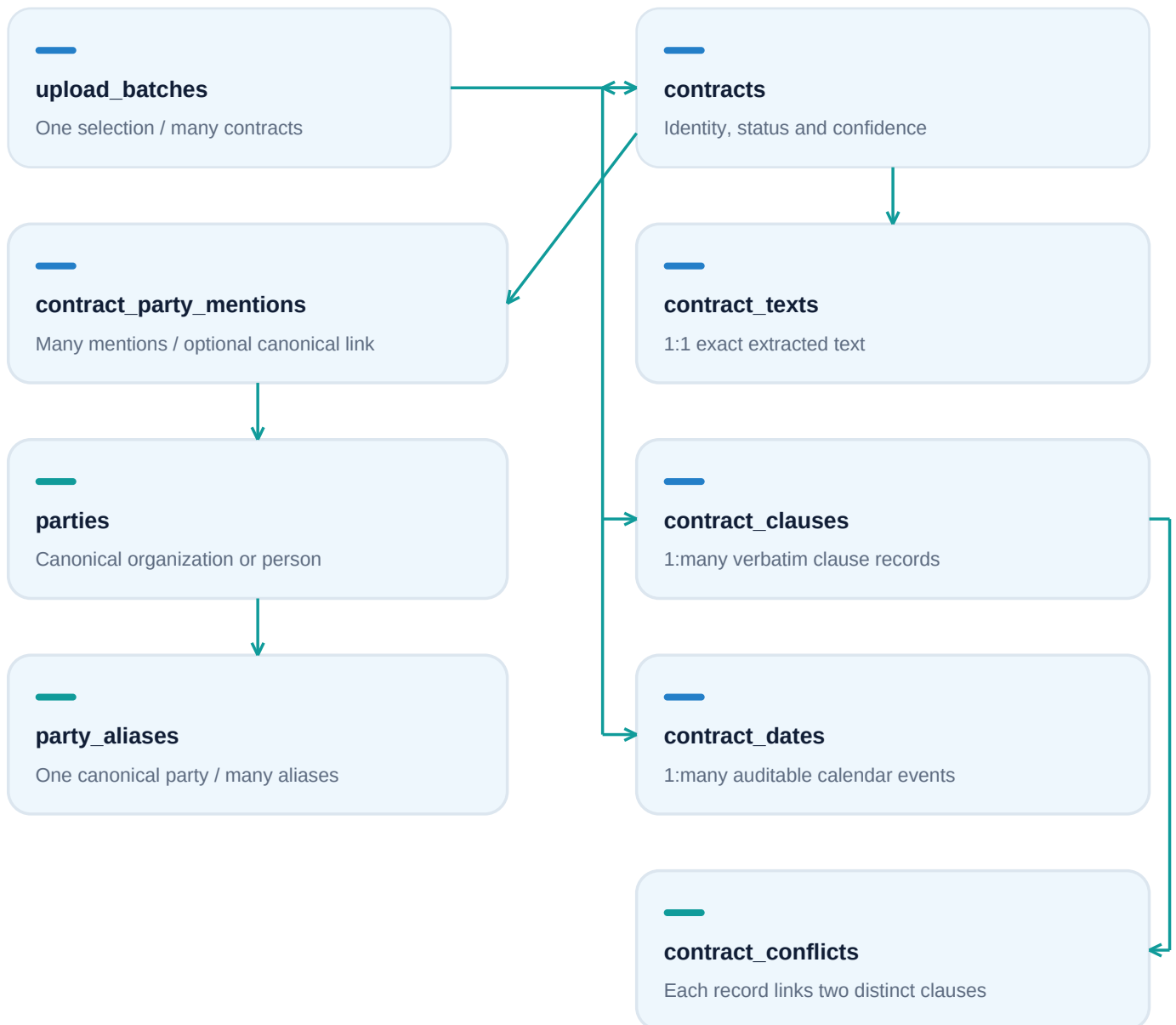


The backend creates temporary file copies only while processing an upload. It persists the formatter's exact extracted text, but not the uploaded binary. The nested JSON returned by the HTTP API is assembled from normalized relational rows.

High-level data flow

- 1. File or folder selection and upload.** The browser accepts individual files, a multi-file selection, or a selected/dropped folder. Each file is sent with its folder-relative path when one exists. Unsupported files and duplicate paths are rejected without entering the extraction pipeline.
- 2. Temporary local copy and text conversion.** The backend gives each upload a temporary server-side file while it is being processed. `text_formatter` dispatches it to the appropriate reader: embedded text is read from PDF, paragraphs and tables are read from DOCX, TXT is read directly, and PNG or JPEG files go through local OCR. Every reader produces one common text and confidence shape, allowing all later stages to ignore the original format.
- 3. Independent contract analysis.** `contract_processor` sends the formatted text through three complementary paths. The CUAD question-answering model finds verbatim clause spans and categories. ContractNER scans the complete document for contracting parties and determines whether each is an organization or person; overlapping CUAD party evidence can increase confidence but is not required. The date path finds explicit dates, uses ContractNER date labels where available and contextual classification otherwise, and may derive an inclusive expiration date from a grounded fixed duration. Dates controlled by an uncertain future action are not derived.
- 4. Normalization and persistence.** The processor returns a single structured contract result containing the source filename, complete extracted text, clauses, party mentions, and date events. The repository layer converts this result into normalized PostgreSQL rows. Party mentions are conservatively linked to canonical parties using unique normalized aliases; ambiguous matches are left unresolved. Date rows retain confidence, provenance, evidence, and source offsets so explicit and derived values can be distinguished and audited.
- 5. Portfolio conflict analysis.** After an upload is persisted, clauses from each new contract are compared with one another. Clauses are also compared across contracts when both contracts resolve to at least one common canonical party. Duplicate CUAD labels over the same verbatim text are collapsed before comparison. The legal NLI model scores each pair in both directions and only stores high-confidence possible contradictions. Final confidence combines the model result, both clause confidences, and shared-party resolution confidence for cross-contract results.
- 6. Failure isolation and cleanup.** Each file is processed in its own nested transaction. A failure creates an auditable failed-contract row instead of cancelling the rest of the selection. After processing, temporary file copies are removed. Model inference and contract handling occur locally; internet access is used only to download model weights when they are not already cached.
- 7. API and frontend consumption.** Flask endpoints reconstruct nested API responses from the relational tables. The Svelte frontend uses them to show parties, lifecycle status, clauses, and links to stored extracted text. Possible conflicts show both clauses verbatim and link back to their source contracts. Calendar rows are converted into an iCalendar feed; the monthly calendar displays that same feed, and the export button downloads it unchanged for use in another calendar application. Consequently, the visible calendar and exported `.ics` file share one source of truth.

Database schema



PostgreSQL creates the following tables through SQLAlchemy:

[upload_batches](#)

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Upload-selection identifier
selection_name	VARCHAR(512)	not null	Folder name or generic selection label
created_at	TIMESTAMPTZ	not null, server default	Upload time

[contracts](#)

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Contract identifier
batch_id	UUID	foreign key, indexed, not null	Parent upload batch
source	VARCHAR(512)	indexed, not null	Filename only
relative_path	TEXT	not null	Folder-relative path supplied by the browser
file_type	VARCHAR(16)	not null	Extension without the leading dot
confidence	DOUBLE PRECISION	not null	Mean clause, party, and date confidence
status	VARCHAR(24)	indexed, not null	complete or failed
error	TEXT	nullable	Processing error for failed files
created_at	TIMESTAMPTZ	not null, server default	Persistence time

Deleting an upload batch cascades to its contracts and every extracted child record.

contract_texts

Column	PostgreSQL type	Constraints	Purpose
contract_id	UUID	primary and foreign key	One-to-one parent contract
text	TEXT	not null	Exact text_formatter output
confidence	DOUBLE PRECISION	not null	Source extraction/OCR confidence
character_count	INTEGER	not null	Display and diagnostics metadata
content_sha256	VARCHAR(64)	indexed, not null	SHA-256 of UTF-8 extracted text
created_at	TIMESTAMPZ	not null, server default	Persistence time

Text is returned only by the contract-detail endpoint. The contract-list and upload responses omit it to avoid transferring every complete contract when a table view only needs metadata.

contract_clauses

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Clause identifier
contract_id	UUID	foreign key, indexed, not null	Parent contract
order_index	INTEGER	not null	Stable extraction/display order
category	VARCHAR(128)	indexed, not null	One configured CUAD clause category
text	TEXT	not null	Verbatim source span
confidence	DOUBLE PRECISION	not null	Per-clause combined confidence

(contract_id, order_index) is unique.

contract_conflicts

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Stable conflict identifier and frontend anchor
clause_a_id	UUID	foreign key, indexed, not null	First verbatim clause
clause_b_id	UUID	foreign key, indexed, not null	Second verbatim clause
confidence	DOUBLE PRECISION	not null	Combined model, clause, and party confidence
model_confidence	DOUBLE PRECISION	not null	Bidirectional legal NLI confidence
model_id	VARCHAR(256)	not null	Hugging Face checkpoint provenance
created_at	TIMESTAMPTZ	not null, server default	Analysis time

The clause pair is unique and must reference two distinct rows. Both foreign keys cascade on deletion, so removing either source contract removes its conflicts. Multiple CUAD categories for the same verbatim span remain available through the clause table and are grouped in the conflict API response.

parties

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Canonical party identifier
canonical_name	VARCHAR(512)	not null	Preferred display name
normalized_name	VARCHAR(512)	indexed, not null	Conservative matching key
entity_type	VARCHAR(32)	not null	organization or person
verification_status	VARCHAR(24)	indexed, not null	Currently unverified for extracted records
created_at	TIMESTAMPTZ	not null, server default	Creation time
updated_at	TIMESTAMPTZ	not null, server default	Last modification time

party_aliases

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Alias identifier
party_id	UUID	foreign key, indexed, not null	Canonical party
alias	VARCHAR(512)	not null	Source-observed display value
normalized_alias	VARCHAR(512)	indexed, not null	Matching key
source	VARCHAR(32)	not null	Alias provenance, currently contract

(party_id, normalized_alias) is unique.

contract_party_mentions

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Mention identifier
contract_id	UUID	foreign key, indexed, not null	Parent contract
party_id	UUID	foreign key, indexed, nullable	Resolved canonical party
order_index	INTEGER	not null	Stable extraction/display order
raw_text	TEXT	not null	Verbatim entity span
extracted_name	VARCHAR(512)	not null	Trimmed display name
normalized_name	VARCHAR(512)	indexed, not null	Conservative matching key
entity_type	VARCHAR(32)	not null	organization or person
extraction_confidence	DOUBLE PRECISION	not null	CUAD and NER combined confidence
resolution_confidence	DOUBLE PRECISION	not null	Confidence in the canonical link
start_offset	INTEGER	nullable	Start in stored contract text
end_offset	INTEGER	nullable	End in stored contract text

(contract_id, order_index) is unique. Deleting a canonical party sets its mention links to NULL; it does not delete contract evidence.

contract_dates

Column	PostgreSQL type	Constraints	Purpose
id	UUID	primary key	Date event identifier
contract_id	UUID	foreign key, indexed, not null	Parent contract
order_index	INTEGER	not null	Stable extraction/display order
event_date	DATE	indexed, not null	Normalized calendar date
event_desc	VARCHAR(256)	indexed, not null	Classified contractual purpose
confidence	DOUBLE PRECISION	not null	Per-date combined confidence
provenance	VARCHAR(16)	indexed, not null	explicit or derived
evidence	TEXT	nullable	Verbatim source evidence for the date
start_offset	INTEGER	nullable	Evidence start in stored contract text
end_offset	INTEGER	nullable	Evidence end in stored contract text

(contract_id, order_index) is unique. The API serializes event_date as DD/MM/YYYY, but it remains a native DATE in PostgreSQL.

The application creates this schema directly from the SQLAlchemy models at startup.

Backend module reference

[backend/extraction/readers/png_reader.py](#)

Exports `read_image(filepath)`. RapidOCR processes PNG, JPG, and JPEG contract images locally. The returned text joins detected lines with newlines. Its confidence is the character-weighted mean of RapidOCR line confidences, or `0.0` when no text is detected.

Output:

```
{"text": str, "confidence": float}
```

[backend/extraction/readers/pdf_reader.py](#)

Exports `read_pdf(filepath)`. `pypdf` reads the embedded text layer of a softcopy PDF; scanned PDFs are intentionally not OCR'd. Non-empty extracted text receives confidence `1.0`, otherwise `0.0`.

[backend/extraction/readers/docx_reader.py](#)

Exports `read_docx(filepath)`. It reads paragraphs and table cells in document order with `python-docx` and ignores images. Non-empty structural text receives confidence `1.0`, otherwise `0.0`.

[backend/extraction/readers/text_formatter.py](#)

Exports `format_text(filepath)`. It dispatches `.txt`, `.docx`, `.pdf`, `.png`, `.jpg`, and `.jpeg` files to the correct reader and adds source, containing only the basename. Unsupported extensions are logged and return `None`.

Output:

```
{"source": str, "text": str, "confidence": float}
```

[backend/extraction/clause_extractor.py](#)

Exports `extract_clauses(formatted_text)`. It runs the extractive question- answering model `Rakib/roberta-base-on-cuad` against 39 clause-focused CUAD categories. Parties and Expiration Date are excluded because the dedicated party and date extractors own those results. Long contracts are processed through overlapping 512-token windows. Answers are reconstructed from character offsets into the input, so clause text is returned verbatim rather than generated. Low-confidence and no-answer results are filtered, and substantially overlapping answers in the same category are deduplicated.

Output:

```
{
  "source": str,
  "confidence": float,
  "clauses": [{"category": str, "text": str, "confidence": float}],
}
```

Each clause confidence is `text confidence * model answer confidence`. The top-level confidence is the mean of retained clause confidences, or `0.0` when there are no clauses.

[backend/extraction/conflict_extractor.py](#)

Exports `extract_conflicts(pairs)`. It runs `epequeno/legal-entailment-deberta-v3-large` locally as a four-way legal NLI cross-encoder. The dependency is pinned to a reproducible model revision with explicit semantic labels. Every clause pair is scored as A to B and B to A; the geometric mean of the two contradiction probabilities prevents input order from deciding the result. Predictions below 0.75 model confidence are omitted. Stored confidence also incorporates both upstream clause confidences and, for cross-contract comparisons, canonical party-resolution confidence.

[backend/extraction/date_extractor.py](#)

Exports `extract_dates(formatted_text)`. The extractor first identifies explicit, complete calendar dates. ContractNER labels effective and termination dates directly when it has a matching grounded span. A local zero-shot classifier, `MoritzLaurer/deberta-v3-base-zeroshot-v2.0`, classifies only dates without a ContractNER label and filters incidental/reference dates.

Accepted forms include numeric DMY, ISO YYYY-MM-DD, and dates with a full English month name. Ambiguous numeric input is always Singapore DMY, so 04/03/2026 means 4 March 2026. For commencement dates, the local extractive QA model `deepset/tinyroberta-squad2` retrieves a verbatim initial-term duration. A second grounded question is asked only after a duration is found, to exclude terms controlled by a future action. Deterministic calendar arithmetic can then add an inclusive expiration date. For example, a three-year term starting 01/07/2026 ends 30/06/2029. Dates dependent on a future action or uncertain event, such as "30 days after delivery", remain ignored.

Output:

```
{
  "source": str,
  "confidence": float,
  "dates": [
    {
      "date": str,
      "event_desc": str,
      "confidence": float,
      "provenance": "explicit" | "derived",
      "evidence": str,
      "start_offset": int,
      "end_offset": int,
    }
  ],
}
```

An explicit date combines source-text confidence with its ContractNER or zero-shot classification confidence. A derived date also incorporates the grounded duration confidence and a derivation discount. The top-level confidence is the mean of retained date confidences, or 0.0 when there are no dates.

[backend/extraction/party_extractor.py](#)

Exports `extract_parties(text, text_confidence=1.0)` and `normalize_party_name(name)`. The local contract-specialized `agilelab-org/Contractner GLiNER` model searches the complete document for verbatim party spans and classifies each span as an organization or person; NLI provides a fallback type only when ContractNER cannot type the span.

The comparison key applies Unicode normalization, case folding, whitespace and punctuation normalization, and normalization, and conservative corporate suffix normalization while retaining the legal suffix. `party_extractor.py` also runs ContractNER over overlapping source-offset-preserving windows and deduplicates overlapping predictions by score.

[backend/extraction/model_registry.py](#)

Owns process-wide lazy loaders for inference models shared by multiple extractors. Model files may be downloaded from Hugging Face on first use, but contract text is always processed locally and is never sent to Hugging Face.

[backend/evaluation.py](#)

Runs party and date extraction against `evaluations/contracts.json` and prints exact-match precision, recall, and F1, plus per-case predictions. Run it with:

```
python -m backend.evaluation
```

The starter corpus includes leases, fixed terms, and an event-dependent end date that must not be derived. Expand this corpus with reviewed Singapore contracts before calibrating thresholds or replacing models.

[backend/persistence/party_resolution.py](#)

Exports `resolve_party_mentions(session, contract, mentions)`. Resolution uses a unique exact normalized alias. A new unverified canonical party is created when an alias is new. Mentions with ambiguous aliases retain a `NULL` party ID.

[backend/extraction/contract_processor.py](#)

Exports `process_contract(filepath, source)`. This orchestration layer runs the formatter and the clause, party, and date extractors. It preserves the uploaded basename rather than the temporary server filename and returns the complete formatted text for persistence.

The contract-level confidence is the equal-weight mean of every extracted clause, party, and date confidence. It is `0.0` when none of those extractors returns a result. The frontend displays that empty case as - and excludes it from the workspace average.

[backend/services/calendar_export.py](#)

Exports `build_calendar(events)`. It converts normalized contract dates into RFC 5545-style all-day VEVENT records, escapes text values, folds content lines to 75 UTF-8 octets, and emits CRLF line endings. Event UIDs are derived from persistent `contract_dates` UUIDs, so repeated exports remain stable.

[backend/services/conflict_analysis.py](#)

Builds eligible candidate pairs, collapses duplicate CUAD categories over identical text, requires a shared canonical `party_id` for cross-contract comparisons, and persists model predictions. Every unique clause pair within a single contract remains eligible even when that contract has no extracted parties. Existing immutable clause pairs are not rescored after later uploads.

[backend/persistence/database.py](#)

Defines the SQLAlchemy engine, session factory, normalized ORM models, API serialization, schema initialization, and metadata. DATABASE_URL defaults to the local Compose database and can be overridden in the environment.

Database check constraints enforce confidence ranges, valid processing states, party entity and verification states, non-negative ordering, and valid mention offsets. An upload batch cannot contain the same relative path twice.

[backend/persistence/contract_repository.py](#)

Owns the conversion of extraction results into normalized ORM graphs, including text hashing, date conversion, child ordering, and party resolution. Routes do not construct persistence models directly.

[backend/config.py](#)

Loads the project .env file and centralizes the database URL, model cache, frontend build directory, and API pagination limits.

[backend/app.py](#)

Defines the Flask application factory, JSON and calendar routes, upload orchestration, and production frontend serving. Database session creation is injectable for tests. Uploads are copied to a temporary directory, processed sequentially, and persisted. A processing error creates a failed contract row with confidence 0.0; unsupported files are returned in the rejected list.

HTTP API

- GET /api/health verifies that the database can answer a query and returns { "status": "ok", "database": "ok" }.
- GET /api/contracts?limit=100&offset=0 returns contracts with nested clauses, parties, dates, and a has_conflicts flag, but omits complete contract text. limit is constrained to 1-500 and offset must be non-negative.
- GET /api/contracts/search?q=termination&limit=20 performs a local fuzzy search over complete stored contract text. It returns ranked passages with filenames, line numbers, match scores, and source offsets. Queries contain 2-200 characters and the result limit is constrained to 1-50.
- GET /api/contracts/{contract_id} returns one contract and its stored text.
- GET /api/conflicts returns every possible contradiction, its combined and model confidences, scope, shared party names, and both verbatim clauses with their source contract IDs, filenames, paths, and CUAD categories.
- DELETE /api/contracts/{contract_id} permanently deletes a contract and its extracted text, clauses, dates, and party mentions. Its upload batch and normalized parties are also removed when they have no remaining references.
- GET /contracts/{contract_id}/text serves the local, styled extracted-text viewer. The viewer retrieves the stored text from the detail endpoint and inserts it as text content, preserving whitespace without interpreting contract content as HTML.
- GET /api/calendar.ics returns all date rows belonging to completed contracts as an inline iCalendar feed.
- GET /api/calendar.ics?download=true downloads that feed as eagle-eye-contract-calendar.ics.
- POST /api/contracts/upload accepts repeated multipart files fields and optional matching paths fields. It returns stored contracts and rejected unsupported files.

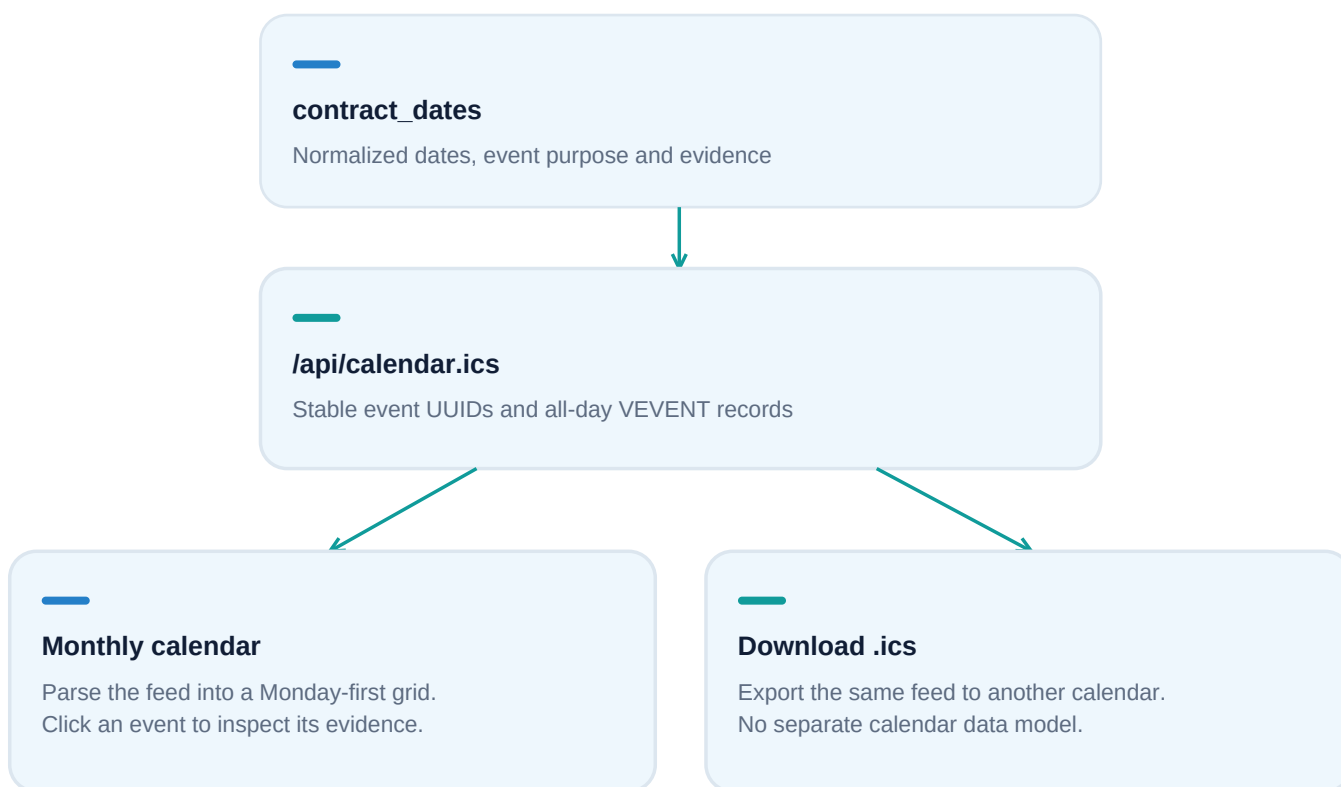
The contract response shape is:

```
{
  "id": str,
  "batch_id": str,
  "source": str,
  "relative_path": str,
  "file_type": str,
  "confidence": float,
  "text": dict | None, # detail endpoint only
  "clauses": list,
  "parties": list,
  "dates": list,
  "status": str,
  "error": str | None,
  "created_at": str | None,
  "has_conflicts": bool,
}
```

Frontend calendar

The frontend fetches `/api/calendar.ics`, unfolds and parses the generated `VEVENT` records, and renders them in a Monday-first monthly grid. Previous, next, and Today controls change the visible month. Up to three events are shown inside a day cell, with an overflow count for additional events. Event colours differentiate starts/effective dates, payment/delivery dates, renewals, termination/end dates, and other deadlines.

The Export `.ics` button downloads the same feed used by the on-screen calendar; there is no separate JSON calendar representation that could drift from the export. All extracted dates are all-day events because the current extraction schema contains calendar dates but no explicit times or time zones. Calendar tooltips and contract term boundaries identify model-derived expiration dates. Clicking an event opens its date, description, contract, confidence, provenance, source evidence, and source-text location. The same metadata is embedded in the iCalendar event through `X-EAGLEEYE-*` properties.



Frontend contracts view

The fixed header uses accessible tab controls for Contract intake, Calendar, Contracts, and Conflicts. Only the selected panel is displayed; the URL hash records the selection, and the tab list supports arrow, Home, and End keyboard navigation.

Contract intake provides separate controls for selecting a complete folder or one or more individual files. Drag-and-drop accepts either form. Tab panels remain mounted while hidden, so an active upload and its progress card survive tab navigation.

The Contracts tab lists processed files. Clicking a row opens a modal containing normalized parties, the identified contract term, and a scrollable list of verbatim clauses with their CUAD categories and confidence scores. Identical clause text is displayed once with every matching category. Clicking the filename instead opens the extracted text in a new browser tab, formatted as a readable document. The details dialog also provides a confirmed delete action that refreshes the contract list, calendar, and active search results after deletion.

Contracts participating in a possible conflict have a red row edge and status badge. Their detail dialogs render implicated clauses in red. Selecting one closes the dialog, switches to the Conflicts tab, and scrolls to the stable conflict card. Each card shows both clauses verbatim, links both source filenames to their extracted-text views, and distinguishes same-contract from shared-party comparisons.

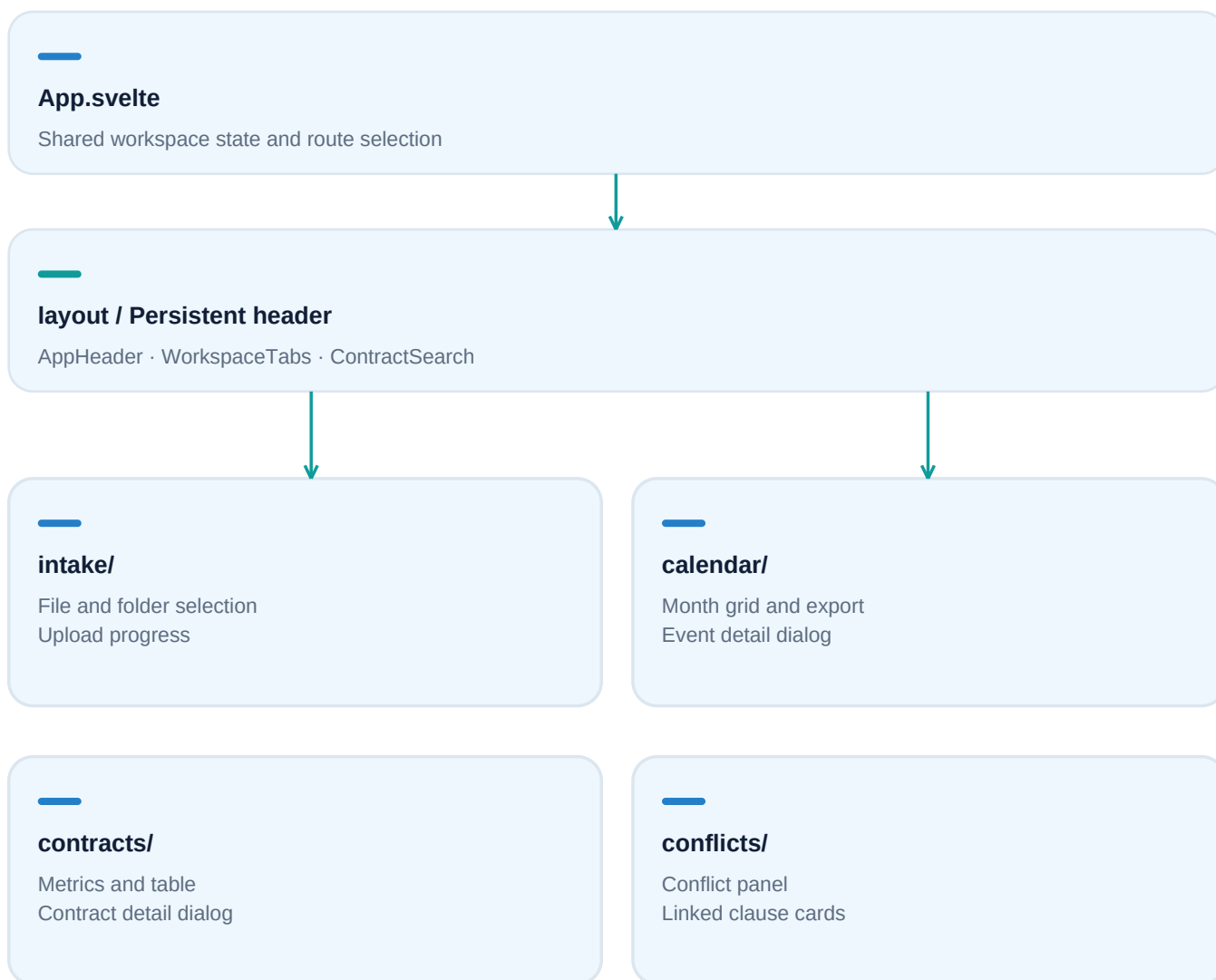
The persistent header search bar searches the complete locally stored text rather than only filenames or extracted clauses. Exact phrases rank first; token and phrase similarity tolerate close spellings and OCR errors. Results appear in a header dropdown with the matching passage, source path, line number, and match percentage. Opening a result launches the text viewer at the stored source offsets and highlights the matching passage.

Lifecycle status is computed against the current date in Asia/Singapore. Contract effective or commencement date is treated as the start boundary; Contract expiration or end date and Termination date are treated as end boundaries. An explicit termination date takes precedence over scheduled expiration dates; otherwise, the latest extracted expiration is used to allow for extensions. End dates are inclusive. A missing start or end is treated as an open interval, but a contract with neither boundary is shown as `Dates unavailable`. Contradictory boundaries are flagged as `Review dates`. Active contracts receive a green visual highlight.

Frontend architecture

The frontend is a Svelte 5 TypeScript single-page application built with Vite and styled with Tailwind CSS. It uses Svelte runes for explicit component state and derived values. During development, Vite serves the frontend on port 5173 and proxies `/api` to Flask on port 8000. A production build is emitted to `frontend/dist`, which Flask serves for the workspace and extracted-text routes.

Components are grouped by the feature that owns their behavior:



Component inventory

App.svelte	shared workspace state and routing
layout/AppHeader.svelte	persistent shell
layout/WorkspaceTabs.svelte	
search/ContractSearch.svelte	
intake/ContractIntake.svelte	selection, drag/drop, upload progress
calendar/CalendarPanel.svelte	month navigation and export
calendar/MonthGrid.svelte	
calendar/CalendarEventDialog.svelte	
contracts/ContractsPanel.svelte	
contracts/ContractMetrics.svelte	
contracts/MetricIcon.svelte	
contracts/ContractTable.svelte	
contracts/ContractDialog.svelte	
conflicts/ConflictsPanel.svelte	
conflicts/ConflictCard.svelte	
text/ContractTextPage.svelte	
shared/Toast.svelte	

`src/lib/api.ts` owns typed HTTP calls. `src/lib/types.ts` defines the shared frontend domain model. `src/lib/contracts.ts` owns confidence and lifecycle calculations, `src/lib/calendar.ts` owns iCalendar parsing, and `src/lib/files.ts` owns browser file/folder traversal. Feature components do not duplicate those rules. User-driven state changes are handled by explicit DOM event handlers; the frontend does not use Svelte effects.

Local models and privacy

Hugging Face model weights for CUAD extraction, legal contradiction NLI, temporal QA and classification, and ContractNER may be downloaded from the internet on first use and are cached in `.models`. Inference happens in the Eagle Eye Python process; contract text is not sent to Hugging Face or another hosted inference API. RapidOCR also runs locally. Normal package/model download metadata may still be visible to the relevant download servers.

Public model hosts

Model weights and normal download metadata

First-use download

Local Eagle Eye environment

Cached model weights in `.models`
Python inference processes the contract text locally.
PostgreSQL stores extracted text and structured evidence.

Retention boundary

Exact extracted text is retained. Original uploaded binaries are temporary.

Prototype limitations

- Extraction is probabilistic and requires human review; it is not legal advice.
- Conflict results are possible textual contradictions, not findings about enforceability or which provision controls. The legal NLI checkpoint is primarily oriented toward US commercial-contract language and truncates a clause pair at 512 tokens.
- Conflict analysis only sees successfully extracted CUAD clauses. Across contracts it also depends on both documents resolving to the same canonical party, so extraction or resolution misses can prevent a comparison.
- Clause extraction performs one CUAD query per category and can be slow on large contracts. Uploads are currently processed synchronously and sequentially.
- Text search scans stored contracts in the application process. A larger deployment should move candidate retrieval to a PostgreSQL trigram or full-text index.
- Scanned PDFs are not OCR'd. Image OCR supports one image file at a time.
- ContractNER is trained primarily on SEC EDGAR contracts, not a representative Singapore corpus. Canonical parties remain unverified until reviewed or checked against a registry.
- Fuzzy party matching and a manual ambiguity-resolution interface are not yet implemented.
- CUAD date-category clauses and structured calendar events are retained independently. The date extractor is authoritative for the calendar because it validates explicit dates and labels deterministic end dates as derived.
- There is no authentication, authorization, upload-size policy, malware scanning, or background job queue yet.
- Full formatted text is retained, but original binaries are not. Re-running a different document parser or OCR engine requires uploading the source files again.